Angular 21 Schulungs-Glossar (Tag 1 – Tag 5)

🧭 Die 9 zentralen Kern-Metaphern des Kurses

Konzept	Bildhafte Metapher für DGS	Technische Bedeutung
Component (Komponente)	**LEGO-Baustein / Box**	Ein sichtbarer, in sich geschlossener Teil der Webseite (z. B. Header, Kundenliste, Kundenkarte). Besteht aus Aussehen (HTML), Verhalten (TypeScript) und Design (CSS).
Signal (Reaktiver Zustand)	**Lämpchen / Sensor / Automatische Glocke**	Ein Datenbehälter. Sobald sich der Inhalt ändert, „leuchtet" er auf und die HTML-Seite aktualisiert den Text an dieser Stelle vollautomatisch, ohne die ganze Seite neu zu laden.
Service & Dependency Injection	**Spezialwerkzeug / Fachabteilung auf Abruf**	Eine Hilfsklasse für Geschäftslogik (z. B. Daten vom Server holen). Komponenten müssen das Rad nicht neu erfinden, sondern „bestellen" den Service per `inject()`.
Routing	**Wegweiser / Umschalten der Bühne**	Die Webseite bleibt immer dieselbe (kein Browser-Reload), aber je nach URL (z. B. `/customer/5`) wird ein anderer Baustein auf die Bühne geholt.
Observable / Stream	**Fließband / Daten-Strom**	Daten, die mit zeitlicher Verzögerung nacheinander eintreffen (z. B. Antworten vom Webserver oder Klicks des Nutzers).
Store (Zustandsspeicher)	**Zentrale Lagerhalle**	Ein einziger Ort, an dem die Daten einer Funktion (z. B. Kundenliste, Filter, Auswahl) liegen. Alle Komponenten holen sich dort Ware und melden Änderungen über feste Methoden.
effect (Nebenwirkung)	**Klingel / Signal nach draußen**	Wenn sich ein Signal ändert, „klingelt" es draußen: etwas außerhalb von Angular wird ausgelöst (z. B. Speichern im Browser-Speicher, Logging). Nicht zum Rechnen gedacht.
Vertrauensgrenze (Sicherheit)	**Grenzkontrolle am Zoll**	Alles, was von außen kommt (Eingaben, URL, Server-Antworten), wird geprüft, bevor es durchgelassen wird. Das Frontend selbst ist nie ein sicherer Ort.
KI-Assistent	**Schneller Assistent ohne Verantwortung**	Die KI schreibt schnell Vorschläge, kann aber irren und klingt trotzdem sicher. Prüfen und Verantworten bleibt beim Menschen.

📅 TAG 1: Grundlagen, Standalone-Architektur, Templates & Signals

Themenschwerpunkt: *Einordnung von Angular, TypeScript, Projektstruktur, Komponenten erstellen, Daten im Template anzeigen, Bedingungen/Schleifen und lokaler Zustand mit Signals.*

1.0 Einordnung: Wo steht Angular?

- **Bibliothek (Library):** Ein Werkzeugkasten für **eine** Aufgabe, z. B. nur das Zeichnen der Oberfläche (React). Alles andere sucht sich das Team selbst aus. Bild: *einzelnes Werkzeug.*
- **Framework** *(Frejm-wörk)*: Ein **fertiger Baukasten** mit Regeln. Angular bringt Routing, Formulare, HTTP, Dependency Injection, Tests und Werkzeug (CLI) gleich mit („Batteries included" = Batterien sind schon eingelegt). Bild: *Möbel-Bausatz mit allen Schrauben.*

- **Meta-Framework** *(Meta-Frejm-wörk)*: Baut auf einem Framework auf und gibt noch mehr vor, z. B. Seiten-Struktur, Server-Rendering und Build. Beispiele: Next.js, Nuxt, Analog.
- **Konvention vor Konfiguration:** Jedes Angular-Projekt ist ähnlich aufgebaut. So findet man sich schnell zurecht.
- **Einseitiger Datenfluss:** Daten fließen **nach unten** (`input`), Ereignisse **nach oben** (`output`). Bild: *Wasser fließt bergab, Meldungen gehen per Telefon hoch.*
- **Typische Einsatzgebiete:** Passt gut für große, langlebige Firmen-Anwendungen mit vielen Teams und vielen Formularen. Passt weniger für reine Text-Seiten (Blogs) oder sehr kleine Widgets.
- **Früher / Heute:** Altes und neues Angular kommen in der Praxis nebeneinander vor. In Bestandsprojekten findet man oft die „alte" Seite.
  - `NgModule` → Standalone-Komponenten
  - Zone.js (automatische Überwachung) → Signals und zoneless
  - `*ngIf`, `*ngFor` → `@if`, `@for`, `@switch`
  - `@Input()`, `@Output()`, `EventEmitter` → `input()`, `output()`, `model()`
  - Konstruktor-Injection → `inject()`
  - Klassen als Guards/Interceptors → Funktionen
  - Karma/Jasmine → Vitest
- **SPA und SSR: SPA** (Single Page Application) = eine Seite, die im Browser umgebaut wird (mehr in 3.1). **SSR** *(Es-Es-Ar)* = Server-Side Rendering: Der Server liefert schon fertiges HTML (mehr in 5.2).
- **Release-Rhythmus:** Angular erscheint mit **zwei Hauptversionen (Major) pro Jahr**. Jede Version wird ca. 18 Monate lang unterstützt. Bild: *feste Fahrplan-Abfahrten.* (Genaue Zahlen: auf angular.dev prüfen.)
- **LTS** *(El-Ti-Es)*: „Long Term Support" = Langzeit-Unterstützung. Eine Version bekommt für längere Zeit Fehlerbehebungen und Sicherheitsfixes.
- **`ng update`:** Der Befehl, der ein Projekt auf die nächste Angular-Version hebt. Er ändert Code automatisch mit (Migrations-Schematics = fertige Umbau-Anleitungen).
- **Angular CLI (`ng`):** Befehle wie `ng new` (neues Projekt), `ng serve`, `ng generate` (Dateien erzeugen), `ng build`, `ng test`, `ng update`.
- **esbuild / Vite** *(I-Es-Bild / Wiet)*: Die schnellen Bau-Werkzeuge unter der Haube. Sie bauen die App und starten den Entwicklungsserver sehr zügig. **HMR** (Hot Module Replacement) = Änderungen erscheinen sofort im Browser, ohne alles neu zu laden.
- **ESLint & Prettier** *(I-Es-Lint, Pri-ti-er)*: ESLint meldet Code-Probleme (Rechtschreibprüfung für Code). Prettier macht den Code einheitlich schön (Formatierung).
- **Angular DevTools** *(Ang-gju-lar Dev-Tuhls)*: Browser-Erweiterung. Zeigt den Komponentenbaum und misst, was wie oft neu geprüft wird (Profiler). Bild: *Röntgenbild der App.*
- **Invoice-App (roter Faden):** Die eine Kundenverwaltungs-App, die in allen Übungen Schritt für Schritt weiterwächst.

1.1 TypeScript & Tooling

- **TypeScript (TS):** Eine Erweiterung von JavaScript. Der Code wird streng typisiert (jede Variable hat einen festen Typ wie Zahl, Text oder Wahrheitswert), um Tippfehler vor dem Ausführen zu verhindern.
- **Typisierung (`string`, `number`, `boolean`, `Date`):**
  - `firstname: string` = Nur Text erlaubt.
  - `age: number` = Nur Zahlen erlaubt.
  - `isActive: boolean` = Nur `true` (wahr) oder `false` (falsch).
- **Interface / Modell (`interface Customer { ... }`):** Ein Bauplan für Datenobjekte. Definiert, welche Felder ein Kunde haben muss (z. B. Vorname, Nachname, ID).
- **CLI (Command Line Interface / Terminal):** Das Text-Befehlswerkzeug. Mit Befehlen wie `ng serve` (Entwicklungsserver starten) oder `ng build` (fertige App bauen) wird Angular gesteuert.

- **Node.js & npm:** Die Ausführungsumgebung auf dem Entwicklerrechner und der Paketmanager, der benötigte Bibliotheken aus dem Internet lädt.

1.2 Standalone Components & Bootstrapping

- **Standalone Component (`standalone: true`):** Moderne Angular-Komponenten sind selbstständig. Sie benötigen keine übergeordneten Module (`NgModule`) mehr und deklarieren ihre benötigten Abhängigkeiten direkt im eigenen `imports: []`-Array.
- **Bootstrapping (`bootstrapApplication(AppComponent)`):** Der Zündschlüssel beim Starten der App in `main.ts`. Lädt die Wurzelkomponente (`AppComponent`) in den Browser.
- **Decorator (`@Component({ ... })`):** Ein Hinweisschild vor einer Klasse, das Angular sagt: „Diese Klasse ist eine UI-Komponente mit HTML-Vorlage und Stylesheet".
- **Selector (`selector: 'app-root'`):** Der eigene HTML-Tag-Name, unter dem die Komponente in anderen Vorlagen verwendet werden kann (z. B. `<app-root></app-root>`).
- **Zoneless (`provideZonelessChangeDetection()`):** Das moderne Angular-System zur Aktualisierung der Bildschirmanzeige rein über Signals, ohne alte Hintergrund-Überwachung (`zone.js`).

1.3 Templates & Datenbindung (Data Binding)

- **Template (HTML-Vorlage):** Die sichtbare Struktur der Komponente.
- **Interpolation (`{{ customer.firstname }}`):** Text-Ausgabe von Variablen im HTML. Bild: *Einsetzen eines Platzhalters in einen Formularbrief.*
- **Property Binding (`[src]="customer.imageUrl"`):** Übergabe von dynamischen Werten aus dem TypeScript-Code an HTML-Attribute (gekennzeichnet durch eckige Klammern `[...]`).
- **Event Binding (`(click)="onSelect()"`):** Reaktion auf Nutzeraktionen wie Mausklicks oder Tastatureingaben (gekennzeichnet durch runde Klammern `(...)`).
- **Two-Way Binding (`[(ngModel)]="customer.firstname"`):** Beidseitige Datenverbindung. Ändert der Nutzer den Text im Eingabefeld, ändert sich die Variable im Code – und umgekehrt. (Merkwort: *Banana in a Box* `[()]`).

1.4 Moderner Built-in Control Flow

- `@if (condition) { ... } @else { ... }`: Bedingte Anzeige. Ein HTML-Block wird nur gerendert, wenn die Bedingung wahr ist (z. B. Daten vorhanden).
- `@for (item of items; track item.id) { ... }`: Schleife/Wiederholung. Wiederholt einen HTML-Abschnitt für jedes Element einer Liste.
- `track`: Ein zwingend erforderlicher Schlüssel (z. B. Kunden-ID), damit Angular bei Änderungen nur genau das veränderte Listenelement neu zeichnet und nicht die ganze Liste (hohe Performance).
- `@empty`: Wird innerhalb von `@for` angezeigt, wenn die Liste leer ist (z. B. „Keine Kunden gefunden").
- `@switch (status) { @case ('active') { ... } }`: Fallunterscheidung für mehrere feste Zustände.

1.5 Signals (Das reaktive Herzstück)

- `signal(initialValue)`: Erzeugt einen reaktiven Datenwert. Auslesen erfolgt mit Funktionsaufruf: `mySignal()`.
- `.set(newValue)`: Überschreibt den Wert eines Signals komplett.
- `.update(fn)`: Berechnet einen neuen Wert basierend auf dem alten (z. B. `count.update(c => c + 1)`).
- `computed(() => ...)`: Ein abgeleiteter Wert (Berechnung), der sich automatisch neu berechnet, sobald sich ein darin verwendetes Signal ändert (z. B. `fullName = computed(() => firstname() + ' ' + lastname())`).
- `effect(() => ...)`: Führt automatisch eine Nebenwirkung aus (z. B. Logging oder Speichern im LocalStorage), wenn sich beobachtete Signals ändern.

1.6 Komponenten-Kommunikation

- **Signal Input (`input<T>()` / `input.required<T>()`):** Datenempfang von der Elternkomponente. Bild: *Der Briefkasten der Kindkomponente.*
- **Output API (`output<T>()`):** Sender für Ereignisse an die Elternkomponente. Ersetzt das frühere `EventEmitter`. Aufruf mit `.emit(daten)`.
- **Deferrable Views (`@defer (on viewport | on idle)`):** Verzögertes Laden von schweren Komponenten (z. B. große Bilder oder Diagramme), erst wenn der Nutzer an die entsprechende Stelle der Seite scrollt.

📅 TAG 2: Formulare, Services, DI, Lifecycle, Pipes & RxJS

Themenschwerpunkt: *Datenbearbeitung, Geschäftslogik in Services auslagern, Datenformatierung mit Pipes und asynchrone Datenströme.*

2.1 Template-Driven Forms (Einfache Formulare, Hauptpfad)

- **`FormsModule`:** Das Angular-Modul für einfache Formulare.
- **`[(ngModel)]`:** Bindet Eingabefelder direkt an Eigenschaften eines Objekts.
- **Form Submission (`(ngSubmit)="onSave()"`):** Das Absenden eines Formulars durch Klick auf einen Speichern-Button.
- **Weiter in Tag 3 (3.2):** Dort kommen Prüfregeln (`required`, `pattern`) und Fehlerzustände (`dirty`, `touched`, `invalid`) für das Bearbeiten-Formular dazu.

2.2 Services & Dependency Injection (DI)

- **Service:** Eine TypeScript-Klasse, die reine Logik und Datenverwaltung enthält (z. B. `CustomerService`). Komponenten sollen schlank bleiben und delegieren Aufgaben an Services.
- **`@Injectable({ providedIn: 'root' })`:** Kennzeichnet eine Klasse als injizierbaren Dienst, der app-weit als Singleton (nur eine einzige Instanz existiert) verfügbar ist.
- **Dependency Injection (DI):** Entwurfsmuster, bei dem eine Komponente benötigte Dienste nicht selbst mit `new Service()` erzeugt, sondern vom Angular-System automatisch „überreicht“ bekommt.
- **`inject(CustomerService)`:** Die moderne Angular-Funktion, um Dienste direkt bei der Variablen-Deklaration anzufordern (Alternative zum alten Konstruktor-Parameter).

2.3 Component Lifecycle (Lebenszyklus)

- **Lifecycle Hooks:** Spezielle Methoden, die Angular zu bestimmten Zeitpunkten im Leben einer Komponente automatisch aufruft.
- **`OnInit` / `ngOnInit()`:** Wird genau einmal aufgerufen, nachdem die Komponente initialisiert und alle Inputs bereitgestellt wurden. Typischer Ort zum ersten Laden von Daten.
- **`OnDestroy` / `ngOnDestroy()`:** Aufräumarbeiten kurz bevor die Komponente vom Bildschirm entfernt wird (z. B. Timer stoppen oder Streams beenden).

2.4 Pipes (Datenformatierung im Template)

- **Pipe (Symbol `|`):** Ein Filter/Transformator im HTML-Template, der Rohdaten für den Benutzer formatiert darstellt, ohne die Originaldaten im Speicher zu verändern.
- **`DatePipe` (`customer.lastOrderDate | date:'mediumDate'`):** Formatiert ein technisches Datum in ein lesbares Format (z. B. `10.10.2026`).
- **`CurrencyPipe` (`price | currency:'EUR'`):** Formatiert Zahlen als Geldbetrag mit Währungssymbol.
- **Custom Pipe (`@Pipe({ name: 'hasLikes', standalone: true })`):** Eine selbst geschriebene Pipe mit einer `transform(value)`-Methode für eigene Formatierungs- oder Filterlogik.

2.5 Asynchronität, RxJS & Signals-Brücke

- **Asynchron:** Vorgänge, die Zeit benötigen und im Hintergrund laufen (z. B. Serveranfragen). Der Browser blockiert dabei nicht.
- **Observable:** Ein Datenstrom aus der Bibliothek **RxJS**. Liefert 0, 1 oder viele Werte über die Zeit.
- **`subscribe()` & `unsubscribe()`:** Anmelden am Datenstrom (Zuhören) und Abmelden (um Speicherlecks zu verhindern).
- **`toSignal(observable$)`:** Die moderne Brückenfunktion in Angular. Wandelt ein RxJS-Observable direkt in ein Angular-Signal um, sodass im Template keine manuelle `async`-Pipe mehr nötig ist.

📅 TAG 3: Routing, Formulare bearbeiten & HTTP-Backend-Anbindung

Themenschwerpunkt: *Mehrseitige Anwendungen (SPA-Routing), ein einfaches Bearbeiten-Formular mit Prüfregeln und echte REST-API-Kommunikation. Reaktive Formulare sind ein Wahlthema (3.4).*

3.1 Routing & Single Page Application (SPA)

- **Single Page Application (SPA):** Eine Webanwendung, die nur aus einer einzigen HTML-Seite besteht. Beim Wechsel von Seiten/Ansichten wird die Webseite nicht neu geladen, sondern Angular tauscht nur Komponenten blitzschnell im Browser aus.
- **`provideRouter(routes, withComponentInputBinding())`:** Aktiviert das Routing-System in `main.ts`.
- **Routes (`app.routes.ts`):** Eine Konfigurationsliste, die URL-Pfade mit Komponenten verknüpft (z. B. `{ path: 'customer', component: CustomerListComponent }`).
- **`<router-outlet></router-outlet>`:** Der dynamische Platzhalter/Rahmen im HTML, an dessen Stelle die zur aktuellen URL passende Komponente gerendert wird.
- **`routerLink="/customer"`:** Das Angular-Äquivalent zum normalen HTML-Link (`<a href>`), verhindert das Neuladen der Seite.
- **Dynamic Routes (`path: 'customer/:id'`):** Routen mit variablem Parameter (z. B. ID des Kunden).
- **Component Input Binding:** Durch `withComponentInputBinding()` wird der URL-Parameter `:id` direkt als Signal-Input `id = input<string>()` in die Zielkomponente injiziert.

3.2 Kunde bearbeiten: Template-Driven Form mit Prüfregeln

- **Hauptpfad:** Zum Bearbeiten eines Kunden nutzen wir das **einfache Template-Driven Form** aus Tag 2 (2.1) weiter. Es wird hier um Prüfregeln ergänzt. Die größere Variante (Reactive Forms) ist ein **Wahlthema** (siehe 3.4).
- **`FormsModule` & `[(ngModel)]`:** Verbinden die Eingabefelder mit den Daten. Jedes Feld braucht einen `name`.
- **Prüfregeln als HTML-Attribute:** `required` (Pflichtfeld) und `pattern="..."` (Text muss zu einem Muster passen). Ohne zusätzlichen Code.
- **Fehlerzustände eines Feldes:** Mit `#name="ngModel"` bekommt man Zugriff auf den Zustand des Feldes:
  - **`invalid`:** Die Prüfregel ist verletzt.
  - **`dirty`:** Der Nutzer hat den Wert **geändert**.
  - **`touched`:** Der Nutzer war **im Feld** und hat es wieder verlassen.
  - Typisch: Fehlermeldung erst zeigen, wenn das Feld `invalid` **und** (`dirty` oder `touched`) ist. Bild: *Rote Karte erst zeigen, wenn jemand wirklich gespielt hat.*
- **Bearbeitungskopie mit `linkedSignal`:** Das Formular arbeitet an einer **Kopie** des Kunden. Das Original bleibt unverändert, bis „Speichern" gedrückt wird. Wechselt der Kunde, wird die Kopie automatisch neu erzeugt (siehe `linkedSignal` in 4.7). Bild: *Arbeiten auf einer Kopie, das Original liegt im Schrank.*
- **Speichern (`(ngSubmit)`):** Sendet die Kopie per `HttpClient` (PUT) an den Server. Nur das Lesen läuft später über die Resource API.
- **Doppelte Prüfung:** Die Prüfung im Browser ist Komfort. Der Server prüft **immer noch einmal** (siehe 5.3).

3.3 HTTP & REST-Backend-Kommunikation

- **REST API:** Eine standardisierte HTTP-Schnittstelle auf dem Webserver zum Austausch von JSON-Daten.
- **HTTP-Methoden:**
  - `GET`: Daten vom Server abrufen (z. B. `GET /api/customer`).
  - `POST`: Neue Daten auf dem Server erstellen (z. B. `POST /api/customer`).
  - `PUT` / `PATCH`: Bestehende Daten aktualisieren.
  - `DELETE`: Daten auf dem Server löschen.
- `provideHttpClient()`: Stellt den `HttpClient`-Dienst in der gesamten App bereit.
- `HttpClient`: Der Angular-Dienst zum Senden von Netzwerkanfragen. Methoden wie `.get<Customer[]>()` geben ein RxJS-Observable zurück.
- **JSON (JavaScript Object Notation):** Das Textformat, in dem Daten zwischen Server und Browser übertragen werden.
- **Dev-Proxy (`dev.proxy.json`):** Ein Entwicklungs-Werkzeug, das Anfragen von `localhost:4200` (Angular) an `localhost:3000` (Backend-Server) weiterleitet, um Browser-Sicherheitsblockaden (CORS) zu vermeiden.

3.4 🔸 Wahlthema Z1: Strictly Typed Reactive Forms (Reaktive Formulare)

Nicht im Hauptpfad. Wahlthema im Ordner `Z1_reactive_forms`, für komplexe Formulare. * **Reactive Forms (`ReactiveFormsModule`):** Ein robuster, code-getriebener Ansatz für komplexe Formulare mit voller TypeScript-Typsicherheit. Das Formular wird im **Code** aufgebaut, nicht im Template. * `FormControl<T>`: Verwaltet den Wert, den Gültigkeitsstatus (Valid/Invalid) und Benutzerinteraktionen (Touched/Dirty) eines einzelnen Eingabefeldes. * `FormGroup`: Bündelt mehrere `FormControl`-Felder zu einem gemeinsamen Formularobjekt. * `FormBuilder` / `NonNullableFormBuilder`: Ein Hilfsdienst (`fb = inject(FormBuilder)`), um Formulare mit weniger Schreibaufwand zu erstellen. * **Validierung (`Validators.required`, `Validators.minLength`):** Prüfregeln für Eingabefelder (z. B. Pflichtfeld oder Mindestlänge). * **Template-Driven vs. Reactive:** Template-Driven = einfach, Regeln im HTML (Hauptpfad). Reactive = mehr Code, dafür mehr Kontrolle und Typsicherheit (Wahlthema).

📅 TAG 4: Guards, Resource API, Komponenten aufteilen, Reaktivität & Zustandsverwaltung

Themenschwerpunkt: *Routen schützen, Daten mit der Resource API laden, Bilder und Elemente gezielt nutzen, Inhalte durchreichen, Komponenten sinnvoll teilen, Reaktivität ohne Fallen, Zustand in einem Store ordnen.*

> Hinweis: Tag 4 und Tag 5 gehören nur zum **Kompaktkurs** (5 Tage). Der Grundkurs endet nach Tag 3.

4.1 Guards & Interceptors (Übung 25)

- **Guard (Türsteher):** Eine Funktion, die vor dem Öffnen einer Seite (Route) entscheidet: „Darf der Nutzer hier rein?“ Wenn nein, wird er z. B. zur Login-Seite geschickt. Aussprache: *Gard.*
- **Funktionaler Guard (`CanActivateFn`):** Der Guard ist heute eine einfache Funktion, keine Klasse mehr. Er holt sich Dienste mit `inject()` (z. B. `AuthService` und `Router`).
- `canActivate: [authGuard]`: So wird der Guard an eine Route gehängt. Ohne Klammern: es wird die Funktion übergeben, nicht ihr Ergebnis.
- `AuthService` **(Demo):** Verwaltet nur ein Signal `isLoggedIn`. Das ist in der Übung eine **Simulation**, keine echte Anmeldung.
- **Wichtig:** Ein Guard verbessert nur die **Bedienung**. Er ist **kein Schutz**. Echte Rechte prüft immer der Server (siehe 5.3).
- **Interceptor (Poststelle / Zollstation):** Eine Funktion, die **jede** HTTP-Anfrage (und Antwort) unterwegs abfängt. Dort kann man protokollieren, Header ergänzen oder Fehler behandeln. Aussprache: *Inter-sep-ter.*

- **Funktionaler Interceptor (`HttpInterceptorFn`):** Eine Funktion `(req, next) => next(req)`. `req` = die Anfrage, `next` = „gib sie weiter".
- `provideHttpClient(withInterceptors([loggingInterceptor]))`**:** So wird der Interceptor in `main.ts` angemeldet.
- `loggingInterceptor`**:** Schreibt Start, Erfolg und Fehler jeder Anfrage in die Konsole. Vorsicht: Keine Tokens oder geheimen Header ausgeben.

4.2 Resource API – Daten laden (Übung 26)

- **Resource (Ressource):** Ein Baustein, der Daten **asynchron lädt** und das Ergebnis als **Signals** bereitstellt. Bild: *Lieferdienst mit Statusanzeige.* Aussprache: *Ri-sors.*
- `httpResource()` *(Ejtsch-ti-ti-pi Ri-sors)*: Lädt Daten direkt per HTTP GET von einer Adresse (z. B. `'api/customer'`). Die Adresse wird als Funktion übergeben. Ändert sich ein Signal darin, lädt die Resource neu.
- `rxResource()` *(Ar-Ex Ri-sors)*: Lädt Daten mit einem RxJS-Observable (z. B. aus einem bestehenden Service). Ändert sich der Parameter (`params`, z. B. die `id`), lädt sie **automatisch neu**. Dafür ist kein `effect` mehr nötig.
- `resource()`**:** Die allgemeine Form, z. B. mit einem Promise statt Observable.
- **Zustände der Resource:** `value()` = die Daten, `isLoading()` = lädt gerade, `error()` = Fehler, `status()` = aktueller Zustand. Damit lassen sich Ladehinweis und Fehlermeldung einfach anzeigen.
- **Lesen vs. Ändern:** Die Resource ist zum **Lesen** gedacht (auch wiederholt). Speichern (POST/PUT) bleibt ein normaler `HttpClient`-Aufruf.
- **Hinweis:** In der Übung ist die Resource API als **experimentell** markiert. Die Schreibweise kann sich in neueren Versionen ändern. Aktuelle Doku auf angular.dev prüfen.

4.3 NgOptimizedImage – schnelle Bilder (Übung 27)

- `NgOptimizedImage` *(En-Dschi Opti-maisd Imidsch)*: Eine Angular-Direktive für Bilder. Sie sorgt für besseres Ladeverhalten und warnt bei Fehlern. Muss in `imports` eingetragen werden.
- `ngSrc`**:** Statt `[src]` schreibt man `[ngSrc]="..."`. Angular setzt das echte `src` selbst.
- `width` **und** `height` **(oder** `fill`**):** Pflichtangaben. So weiß der Browser vorher, wie viel Platz das Bild braucht, und die Seite „springt" nicht (siehe **CLS**, 5.2). Ohne diese Angaben gibt Angular einen Fehler aus.
- `priority` *(prai-o-ri-ti)*: Markiert das **wichtigste** Bild, das sofort sichtbar sein soll (above the fold = ohne Scrollen sichtbar). Nur sparsam benutzen, meist höchstens ein Bild pro Seite.
- **Lazy Loading (Bilder):** Bilder weiter unten werden erst geladen, wenn man hinscrollt (siehe 5.2).

4.4 viewChild – Zugriff auf Elemente im eigenen Template (Übung 28)

- `viewChild()` *(Wju-Tschaild)*: Gibt ein **Signal** zurück, das auf ein Element oder eine Komponente im **eigenen** Template zeigt. Aufruf wie bei Signals: `searchInput()`.
- **Signal Query** *(Signal Kwiri)*: Sammelname für die signal-basierten Abfragen `viewChild`, `viewChildren`, `contentChild`, `contentChildren`. Bild: *Suchauftrag an das Template: „Finde mir dieses Element."*
- **Template-Referenzvariable (`#searchInput`):** Ein Namensschild im HTML. Der Text in `viewChild('searchInput')` muss genau zum Namensschild passen.
- `ElementRef` **/** `nativeElement`**:** Ein Verweis auf das echte HTML-Element im Browser. Damit geht z. B. `nativeElement.focus()` (Cursor ins Feld setzen).
- **Wert erst später:** Direkt am Anfang ist der Wert `undefined`. Das Element gibt es erst, nachdem die Seite gezeichnet wurde.
- `viewChild.required()`**:** Für Pflicht-Elemente, die immer da sein müssen.
- `contentChild()`**:** Wie `viewChild`, aber für Elemente, die von **außen eingefügt** wurden (siehe 4.5).
- Mit Bedacht nutzen: Direkter Zugriff auf das DOM ist nur die Ausnahme (Sicherheit: siehe 5.3).

4.5 Content Projection – Inhalte durchreichen (Übung 29)

- **Content Projection** *(Kon-tent Pro-dschek-schen)*: Eine Komponente bekommt von **außen** Inhalt, der in ihr angezeigt wird. Bild: *Bilderrahmen: Der Rahmen bleibt gleich, das Bild darin wechselt.*
- `<ng-content>`: Der **Platzhalter** (Slot) in der Komponente, an dem der fremde Inhalt erscheint.
- **Default-Slot:** Ein `<ng-content></ng-content>` ohne Zusatz fängt alles auf, was sonst nirgends passt.
- **Multi-Slot (`select="[card-header]"`):** Mehrere Plätze mit Namen, z. B. Kopf, Hauptteil, Fuß. Der Inhalt wählt seinen Platz über ein Attribut (`<h3 card-header>`).
- `CardComponent` **(Übung):** Eine wiederverwendbare Karte mit Kopf, Inhalt und Fußbereich.
- **Komposition:** Kleine Bausteine werden **zusammengesteckt** (statt Vererbung). Content Projection ist ein Werkzeug dafür. Bild: *LEGO.*
- Vorteil: Weniger **Prop Drilling** (Daten durch viele Ebenen reichen, siehe 4.6).

4.6 Komponenten aufteilen & Wartbarkeit (Kapitel 1, Übung 31)

- **Wartbarkeit:** Wie leicht lässt sich der Code später lesen, ändern und testen? Ziel: **Änderungen bleiben lokal.**
- **Single Responsibility** *(Single Ri-spon-si-bi-li-ti)*: „Eine Aufgabe pro Baustein." Eine Komponente hat **einen Grund zur Änderung**. Test: Kann ich in einem Satz sagen, was sie tut, ohne „und"?
- **God Component** *(Gott Komponent)*: Eine Riesen-Komponente, die alles macht (Daten, Logik, Layout). Schwer zu verstehen und zu testen.
- **Zu feine Teilung:** Das andere Extrem (eine Komponente pro `<span>`). Der Überblick geht verloren.
- **Prop Drilling** *(Prop Drilling)*: Daten werden durch viele Ebenen durchgereicht, obwohl nur die letzte sie braucht. Besser: Service oder Content Projection.
- **Verdeckte Kopplung:** Ein Kind greift heimlich auf den Eltern-Baustein zu. Das macht Änderungen riskant.
- **Container-Komponente („smart")** *(Kon-tej-ner)*: Kennt Services, lädt Daten und gibt sie nach unten weiter. Bild: *Der Chef, der organisiert.*
- **Darstellungskomponente („dumb")**: Kennt nur `input()` und `output()`. Zeigt an und meldet Ereignisse. Gut wiederverwendbar und leicht testbar. Bild: *Schaufenster ohne Lager.*
- **Schnittstelle (Inputs/Outputs):** `input.required<T>()` für Pflichtdaten. Typen exportieren, kein `any`. Outputs als Ereignis benennen (`customerSelected`). Eingaben im Kind nicht verändern, sondern per Output melden.
- `model()`: Ein Signal mit Zwei-Wege-Verbindung zwischen Eltern und Kind. Bewusst und sparsam einsetzen.
- **Komposition statt Vererbung:** Wiederverwendung durch Zusammenstecken (`<ng-content>`, Direktiven, kleine Komponenten). Vererbung zwischen Komponenten vermeiden.
- **Host-Direktive (`hostDirectives`):** Eine Direktive, die eine Komponente „mitbringt", um gemeinsames Verhalten zu teilen.
- **Trennung von UI und Fachlogik:** Die Komponente zeigt an. Fachregeln stehen im Service/Store. Reine Berechnungen in einer **pure Funktion** oder Pipe (ohne Angular testbar).
- **Abhängigkeitsrichtung:** Abhängigkeiten zeigen **nur nach unten**: Komponente → Service/Store → API-Client. Der Service kennt keine Komponente. Ebenso: `shared` ← `feature`, nie umgekehrt.
- **Feature-Ordner:** Dateien nach Funktion ordnen (`customer/`, `auth/`, `shared/`), nicht nach Typ (alle Services in einem Ordner).
- `strict` **(TypeScript):** Strenger Modus. Kein `any` ohne Begründung.
- **ADR** *(Ei-Di-Ar)*: Architecture Decision Record = kurze Notiz „Warum haben wir das so entschieden?"
- **Faustregel Testbarkeit:** Ist ein Test schwer zu schreiben, ist oft die Struktur das Problem.

4.7 Reaktivität & Datenfluss (Kapitel 2, Übung 32)

- **Quelle → Ableitung → Wirkung:** Das Grundmodell. **Quelle** = `signal` (hält Zustand). **Ableitung** = `computed` (rechnet daraus). **Wirkung** = Template oder `effect` (zeigt an oder greift nach außen). Bild: *Quelle, Fluss, Mühlrad.*
- **Abhängigkeitsgraph:** Angular merkt sich automatisch, wer welches Signal liest. Nichts muss von Hand abonniert werden.

- **Lokaler (Quell-)Zustand:** Kann der Nutzer oder ein Ereignis direkt ändern (z. B. Filtertext, Auswahl).
- **Abgeleiteter Zustand:** Folgt aus anderem Zustand. **Nicht speichern, sondern berechnen** (`computed`).
- **Single Source of Truth** *(Single Sors of Truth)*: „Einzige Quelle der Wahrheit." Jeder Fakt wird nur an **einer** Stelle gespeichert. Sonst laufen Kopien auseinander.
- `computed` **im Detail:**
  - **Lazy** *(Lei-si)*: rechnet erst, wenn jemand liest.
  - **Memoized** *(Mem-o-aisd)*: merkt sich das Ergebnis, solange sich nichts ändert.
  - **Rein:** keine Nebenwirkung. Auf Signals schreiben ist in `computed` verboten (Fehler `NG0600`).
  - Gleichheit per `===`: Als „geändert" gilt nur eine **andere Referenz**.
- `linkedSignal()` *(Linkt-Signal)*: Ein abgeleiteter Wert, den man **trotzdem überschreiben** kann. Beispiel: Die Auswahl springt auf das erste Element, wenn die Liste wechselt. Der Nutzer kann sie aber ändern. Ersetzt das alte Muster „effect setzt ein Signal zurück".
- **Nebenwirkung (Side Effect)** *(Said I-fekt)*: Etwas verändert sich **außerhalb** des Signal-Graphen (Speicher, Logging, Zeichnen in einer Drittbibliothek).
- `effect()` *(I-fekt)*: Führt eine Nebenwirkung aus, wenn sich gelesene Signals ändern. Gute Gründe: `localStorage` sichern, Logging, Drittbibliothek versorgen, DOM-Zugriff.
- **Wann kein** `effect`**?** Wert berechnen → `computed`. Signal zurücksetzen → `linkedSignal`. Auf Klick reagieren → Event-Handler. Daten laden bei neuer `id` → `resource`. Faustregel: **Schreibt ein effect ein Signal, gibt es fast immer eine bessere Lösung.**
- **Wie ein** `effect` **läuft:** Mindestens einmal. Danach bei Änderung. **Asynchron** (nicht sofort nach `set()`). Mehrere Änderungen ergeben **einen** Lauf (**Batching** *(Bät-sching)*).
- `onCleanup`**:** Aufräum-Funktion im effect (z. B. Timer stoppen, bevor er neu startet).
- **Injection Context** *(In-dschek-schen Kon-tekst)*: Der Bereich (Konstruktor oder Feld-Initialisierung), in dem `inject()` und `effect()` erlaubt sind.
- `untracked()` *(An-trakt)*: Liest ein Signal, **ohne** davon abhängig zu werden. Bild: *Heimlich nachschauen, ohne sich für Benachrichtigungen anzumelden.*
- **Mutation vs. neue Referenz** *(Mjuh-tei-schen)*: Ein Array mit `push` zu ändern bemerkt das Signal **nicht** (gleiche Referenz). Richtig: neues Array erzeugen, z. B. `list.update(l => [...l, c])`. Bild: *Nicht im alten Brief radieren, sondern einen neuen schreiben.*
- **Zyklus / Kette:** Ein effect schreibt in ein Signal, ein anderer effect liest es und schreibt zurück. Endlosschleife und kaum zu debuggen. Besser: **eine** Quelle und `computed`.
- **Methoden im Template:** Eine normale Methode läuft bei jeder Change Detection. Ein `computed` nur bei geänderten Abhängigkeiten.
- `toObservable()`**:** Gegenstück zu `toSignal()`. Macht aus einem Signal einen RxJS-Strom (z. B. für Debounce).
- `toSignal` **ohne** `initialValue`**:** Ergebnis ist `T | undefined`. Fehler im Observable werden beim Lesen des Signals erneut geworfen, darum vorher `catchError`.
- **Faustregel RxJS vs. Signals:** Für **Zeit und Ereignisströme** (debounce, switchMap, Retry) RxJS. Für **Zustand** Signals.
- **Zu grobes Signal:** Ein großes Objekt-Signal macht bei jeder kleinen Änderung **alle** Leser ungültig. Besser mehrere kleine Signals plus `computed`.
- **Aktionen:** Änderungen nur über **benannte Methoden** (z. B. `addCustomer`), nicht überall `set()`.

4.8 Anwendungsstruktur & Zustandsverwaltung (Kapitel 3, Übung 33)

- **Zustandsverwaltung (State Management):** Wie und wo eine App ihre Daten hält und ändert. Wächst mit der App: Phase 1 `signal()`, Phase 2 Service, Phase 3 Store-Muster, Phase 4 Redux-Muster (NgRx). Nicht zu früh, nicht zu spät.
- **Drei Schichten: Zustand** (Daten halten, ableiten) – **Logik** (Regeln, Befehle, Laden) – **Darstellung** (anzeigen, Eingaben weitergeben). Komponenten **lesen** Signals und **rufen** Methoden auf.

- **Store (Zentrale Lagerhalle)** *(Stohr)*: Eine Klasse, die Zustand, Ableitungen und Änderungs-Methoden eines Features bündelt. Alle Komponenten holen die Daten von dort.
- **Service mit Signals:** Die einfache Stufe. Ein `@Injectable`-Service hält private Signals und zeigt nach außen nur lesbare Teile.
- **`asReadonly()`:** Macht ein Signal von außen **nur lesbar**. Geändert wird nur über Methoden des Stores (Kapselung).
- **`CustomerStore` (Übung):** Hält `customers`, `filter`, `selectedId`. Leitet `filtered` und `selected` ab. Methoden `load()`, `setFilter()`, `select()`.
- **`LoadState` (Lade-Zustand):** Ein Typ mit festen Zuständen `idle`, `loading`, `error`, `loaded`. Unmögliche Kombinationen (laden **und** Fehler) gibt es nicht.
- **Server-Daten vs. UI-Zustand:** Server-Daten (geladen, haben ein Alter, können Fehler haben) getrennt von UI-Zustand (Dialog offen, Tab gewählt).
- **Normalisierung** *(Nor-ma-li-zie-rung)*: Objekte werden einmal nach `id` abgelegt (Tabelle). Listen enthalten nur die IDs. Bild: *Karteikasten mit Nummern.*
- **Ableiten statt kopieren:** „Ausgewählter Kunde" ist ein `computed` aus `selectedId` und `customers`.
- **Signal Store** *(Signal Stohr)*: Der `signalStore` aus `@ngrx/signals`. Fertige Bausteine: `withState`, `withComputed`, `withMethods`, `patchState`. Erweiterbar (z. B. `withEntities`, `rxMethod`). Installieren: `npm install @ngrx/signals`.
- **NgRx** *(En-Dschi-Ar-Ex)*: Die bekannte Zustands-Bibliothek für Angular nach dem Redux-Muster. Mehr Aufwand, dafür Action-Log, **Time-Travel** (Zustände zurückspulen, *Taim-Trä-wel*) und klare Regeln. Sinnvoll bei sehr großen Apps und vielen Teams.
  - **State:** der Zustand (im Signal Store: `withState`).
  - **Action** *(Äk-schen)*: Die **Absicht** („Kunde hinzufügen"). Im Signal Store: eine Methode.
  - **Reducer** *(Ri-djuh-ser)*: Rechnet aus altem Zustand und Action den **neuen Zustand** (im Signal Store: `patchState`).
  - **Selector** *(Si-lek-tor)*: Eine **Ableitung** aus dem Zustand (im Signal Store: `withComputed`).
  - **Effect** *(I-fekt)*: Eine **asynchrone Nebenwirkung** wie Server-Aufruf (im Signal Store: Methode oder `rxMethod`). Nicht verwechseln mit Angulars `effect()`.
- **Wechselkriterien:** Service + Signals reicht bei 1–3 Komponenten und wenig Nebenwirkungen. Ein Store lohnt sich bei vielen Konsumenten, zusammenhängenden Zuständen, nötiger Nachvollziehbarkeit und mehreren Teams. Merksatz: **Starte einfach, strukturiere, wenn ein Kriterium schmerzt.**
- **Typische Fehlentscheidungen:** Alles in den Store (auch Tooltip-Zustand). **God Store** (ein Store für die ganze App). **Doppelte Wahrheit** (Daten im Store und in der Komponente). Server-Cache mit Client-Zustand verwechseln.
- **Testbarkeit:** Ein Store ist ohne Oberfläche testbar. Methode aufrufen, Signal prüfen.

📅 TAG 5: Tests, Performance, Sicherheit, KI-gestützte Entwicklung & Abschlussprojekt

Themenschwerpunkt: *Code mit Tests absichern, die App schnell machen, Sicherheitslücken im Frontend vermeiden, KI-Vorschläge kritisch prüfen und alles im Abschlussprojekt anwenden.*

5.1 Testbarkeit & Qualität (Kapitel 4, Übung 34)

- **Test:** Ein kleines Programm, das prüft, ob anderer Code richtig arbeitet. Bild: *Sicherheitsnetz beim Umbauen.* Tests sind selbst Code und brauchen Pflege.
- **Testpyramide:** Viele schnelle Tests unten, wenige langsame oben.
- **Unit-Test** *(Juh-nit-Test)*: Prüft eine **kleine Einheit** (Service, Store, pure Funktion). Schnell, viele davon.
- **Komponententest:** Prüft Template und Klasse einer Komponente zusammen (mit `TestBed`).

- **E2E-Test** *(I-tu-I)*: End-to-End. Ein Roboter bedient die **ganze App** im Browser (z. B. mit Playwright oder Cypress). Wenige Tests, nur für wichtige Abläufe.
- **Vitest** *(Wih-test)*: Das Test-Werkzeug (Test-Runner), das in Angular 21 Standard ist. Löst Karma/Jasmine ab. Bausteine: `describe`, `it`, `expect`, `beforeEach`.
- **`vi.fn()`, `vi.spyOn()`, `vi.useFakeTimers()`:** Vitest-Hilfen für Mock-Funktionen, Beobachter und künstliche Uhr.
- **TestBed** *(Test-Bed)*: Der „Prüfstand" von Angular. Baut für den Test eine kleine Angular-Umgebung auf (Dienste, Komponenten).
- **`TestBed.inject(...)`:** Holt einen Dienst im Test.
- **Arrange – Act – Assert** *(Ä-reindsch – Äkt – Ä-sört)*: Das Muster für Tests: 1. Vorbereiten, 2. Ausführen, 3. Ergebnis prüfen. Ein Test prüft **ein** Verhalten.
- **Testdouble (Ersatz-Objekte):** Damit man eine Einheit **isoliert** testet, ersetzt man ihre Abhängigkeiten:
  - **Stub** *(Stabb)* / **Fake** *(Feik)*: einfache Ersatzimplementierung mit festen Daten.
  - **Mock** *(Mock)*: prüft zusätzlich, **ob und wie** aufgerufen wurde (`vi.fn`).
  - **Spy** *(Spai)*: beobachtet ein **echtes** Objekt.
- **Ersatz über `providers`:** Dank Dependency Injection tauscht man einen Dienst im Test einfach aus (`{ provide: CustomerApi, useValue: ... }`).
- **Over-Mocking:** Zu viel ersetzt. Dann testet man nur noch die Ersatz-Objekte.
- **`HttpTestingController`** *(Ejtsch-ti-ti-pi Testing Kontroller)*: Fängt HTTP-Aufrufe im Test ab und liefert selbst gewählte Antworten (`expectOne`, `flush`, `verify`). Dazu `provideHttpClientTesting()`.
- **`fixture.componentRef.setInput(...)`:** So setzt man Signal-Inputs in Tests. Direktes Zuweisen geht nicht. *Fixture = Fiks-tscher*, die Test-Hülle um die Komponente.
- **`await fixture.whenStable()`:** Wartet, bis das Zeichnen fertig ist (wichtig bei zoneless).
- **`TestBed.tick()`:** Führt wartende `effect`s im Test aus.
- **Coverage** *(Kaw-ridsch)*: Testabdeckung. Zeigt, wie viel Code die Tests durchlaufen. Ein **Hinweis, kein Ziel**: 100 % ohne echte Prüfungen beweisen nichts.
- **Guter Test:** Schlägt nur fehl, wenn das Verhalten kaputt ist. Unabhängig, schnell, deterministisch (keine echte Uhr, kein Netz). Prüft **Verhalten**, nicht Interna.
- **Generierte Tests:** Von `ng generate`, IDE oder KI erzeugt. Typische Schwächen:
  - **`toBeTruthy`-Test** („should create"): prüft fast nichts.
  - **Tautologie:** Der Test wiederholt den Code. Beide ändern sich gemeinsam, nichts fällt auf.
  - **Happy Path** *(Hä-pi Paff)*: Nur der Normalfall wird geprüft. Randfälle fehlen (leer, `null`, Fehler).
  - **Snapshot-Test** *(Snäp-schott)*: Vergleicht mit gespeichertem Abbild. Bricht oft und wird blind aktualisiert.
- **Mutationstest von Hand:** Code absichtlich kaputt machen. Wird kein Test rot, ist er wertlos.
- **Verantwortung:** Wer den Test committet, haftet dafür, nicht das Werkzeug.

5.2 Performance & Laufzeitverhalten (Kapitel 5, Übung 35)

- **Performance (Leistung):** Wie schnell fühlt sich die App an? Regel Nr. 1: **Erst messen, dann optimieren.**
- **Ladezeit vs. Laufzeit:** Ladezeit = wie schnell ist etwas sichtbar? Laufzeit = wie flüssig reagiert die App?
- **Core Web Vitals** *(Kor Web Wai-tels)*: Drei Messwerte von Google für das Nutzergefühl:
  - **LCP** *(El-Si-Pi)* – Largest Contentful Paint: Wann ist der Hauptinhalt sichtbar?
  - **INP** *(Ai-En-Pi)* – Interaction to Next Paint: Wie schnell reagiert die App auf Eingaben?
  - **CLS** *(Si-El-Es)* – Cumulative Layout Shift: Springt das Layout? (Bild: *Der Knopf wandert kurz vor dem Klick weg.*)
- **Messwerkzeuge:** Chrome DevTools (Performance, **Flamegraph** *Fleim-Graf*, Long Tasks), **Lighthouse** *(Lait-haus)* für Ladeverhalten, **Angular DevTools Profiler** *(Pro-fai-ler)* zeigt, welche Komponente wie oft geprüft wird.

- **Change Detection** *(Tscheindsch Di-tek-schen)*: Angular prüft, was sich geändert hat, und zeichnet es neu. Früher mit Zone.js den **ganzen Baum**. Heute (zoneless + Signals) nur die **betroffenen Komponenten**.
- `ChangeDetectionStrategy.OnPush` *(On-Pusch)*: Eine Komponente wird nur geprüft, wenn sich ein Input, ein Ereignis aus ihrem Template oder ein gelesenes Signal ändert. Empfohlen für alle neuen Komponenten.
- **Kleine Aktualisierungsbereiche:** Kleine Komponenten mit eigenen Signals. Dann wird nur ein kleiner Teil neu gezeichnet. Ein Grund mehr fürs Aufteilen.
- `track` **in** `@for`**:** `track c.id` (stabile ID) statt `track c` (Objekt). Sonst baut Angular nach dem Neuladen **alle** Zeilen neu, und Zustand wie Fokus geht verloren. `track $index` nur bei statischen Listen.
- **Pure Pipe:** Rechnet nur neu, wenn sich die Eingabe ändert. **Unreine Pipe** (`pure: false`) läuft ständig, mit Vorsicht nutzen.
- **Virtual Scrolling** *(Wör-tschu-el Skrol-ling)*: Bei sehr langen Listen sind nur die **sichtbaren** Zeilen im Dokument. Bild: *Fenster mit Fließband: nur sichtbarer Teil liegt bereit.* Aus `@angular/cdk` (`cdk-virtual-scroll-viewport`, `*cdkVirtualFor`). `itemSize` muss zur Zeilenhöhe passen.
- **Paginierung / Server-seitiges Filtern:** Weniger Daten werden übertragen (Seitenweise).
- **Debounce** *(Di-baunß)*: Wartet nach einer Eingabe kurz (z. B. 250 ms), bevor gerechnet wird. Bild: *Erst ausreden lassen, dann antworten.*
- **Lazy Loading** *(Lei-si Loh-ding)*: Nur laden, was gebraucht wird. Bei Routen: `loadComponent: () => import(...)`. Der Code kommt erst bei Aufruf der Seite.
- `@defer` *(Di-för)*: Schwere Teile der Seite werden erst bei Bedarf geladen (`on viewport`, `on interaction`, `on idle`, `when ...`). Mit `@placeholder` (Platzhalter) und `@loading` (Ladeanzeige).
- **Bundle** *(Bandl)*: Das fertig gebaute Paket mit dem App-Code, das der Browser laden muss. Je kleiner, desto schneller der Start.
- **Tree-Shaking** *(Trie-Schei-king)*: Der Build wirft ungenutzten Code weg. Bild: *Baum schütteln, trockene Blätter fallen ab.*
- **Budget** *(Badd-schet)*: Eine festgelegte Obergrenze für die Bundle-Größe in `angular.json`. Wird sie überschritten, schlägt der Build fehl.
- `ng build --stats-json`**:** Erzeugt eine Statistik-Datei, mit der ein Analyzer zeigt, was im Bundle steckt.
- **Rendering-Strategien:**
  - **CSR** *(Si-Es-Ar)*: Client-Side Rendering. Der Browser baut alles. Einfach, erster Inhalt später.
  - **SSR** *(Es-Es-Ar)*: Server-Side Rendering. Der Server liefert fertiges HTML. Schneller erster Inhalt, gut für Suchmaschinen (SEO).
  - **Hydration** *(Hai-drei-schen)*: Der Browser „belebt" das vom Server gelieferte HTML wieder mit Angular. Bild: *Trockenes Pulver mit Wasser aktivieren.*
  - **Prerendering / SSG** *(Pri-Ren-de-ring)*: HTML wird schon beim **Bauen** erzeugt. Gut für statische Seiten.
  - Für interne Fachanwendungen reicht oft CSR. Für öffentliche Seiten SSR prüfen.
- **Speicherleck (Memory Leak):** Nicht abgemeldete Subscriptions, Timer oder Listener belegen weiter Speicher.

5.3 Sicherheit im Frontend (Kapitel 6, Übung 36)

- **Grundsatz:** Das Frontend ist **nicht vertrauenswürdig**. Der Code läuft im Browser des Nutzers. Er kann ihn lesen, ändern, umgehen.
- **Folgen:** Keine Geheimnisse im Frontend (Schlüssel, Passwörter). Rechte werden im **Backend** geprüft. Guards sind nur Bedienkomfort. Prüfung im Client ist **UX**, Prüfung im Server ist **Sicherheit**.
- **Vertrauensgrenze (Grenzkontrolle)** *(Trust Boundary)*: Die Linie zwischen „eigener Code" und „von außen". Alles von außen ist **Eingabe** und wird geprüft: Formulare, URL-Parameter, Server-Antworten (auch vom eigenen Backend), `localStorage`, Cookies, Uploads, Drittanbieter.
- **XSS** *(Iks-Es-Es)* – **Cross-Site Scripting** *(Kross-Sait Skrip-ting)*: Ein Angreifer schleust Script-Code in die Seite ein. Dieser läuft mit den Rechten des Nutzers (z. B. Token auslesen). Bild: *Ein Fremder schiebt einen Zettel mit Befehlen ins Formular.*

- **Escaping:** Zeichen werden so umgewandelt, dass sie als Text gelten, nicht als Code. Die Interpolation `{{ }}` macht das automatisch.
- **Sanitizing** *(Sä-ni-tai-sing)*: „Reinigen". Bei `[innerHTML]` entfernt Angular gefährliche Teile (`<script>`, Event-Handler). Bei `[href]` / `[src]` werden gefährliche Adressen wie `javascript:` entschärft.
- `DomSanitizer.bypassSecurityTrust...` *(Bai-päss Si-kjuh-ri-ti Trast)*: Schaltet Angulars Schutz **ab** (`...Html`, `...Url`, `...Script`, `...ResourceUrl`). Nur für selbst kontrollierte Inhalte, **nie** für Nutzereingaben. Gilt als **Review-Alarm** und braucht eine Begründung im Code.
- **Weitere Schutz-Umgehungen:** Direkter DOM-Zugriff (`nativeElement.innerHTML`), `document.write`, `eval`, `new Function`, Templates aus Strings, Drittbibliotheken ohne Sanitizing.
- **Token-Speicherort:** `localStorage` und Cookie ohne `HttpOnly` sind bei XSS per Script auslesbar.
- **HttpOnly-Cookie** *(Ejtsch-ti-ti-pi-Ohn-li)*: Ein Cookie, an das Script nicht herankommt. Zusätzlich `Secure` und `SameSite`. Das Backend setzt es. Bevorzugte Variante. Login besser mit bewährten Bibliotheken/Identity Providern als im Eigenbau. (Der `AuthService` aus Übung 25 ist nur Demo.)
- **CSRF / XSRF** *(Si-Es-Ar-Ef / Iks-Es-Ar-Ef)* – **Cross-Site Request Forgery**: Eine fremde Seite löst mit dem Browser des Nutzers (samt Cookies) Aktionen bei deinem Backend aus. Gegenmittel: `SameSite`-Cookies und **CSRF-Token**. Angulars `HttpClient` liest das Cookie `XSRF-TOKEN` und schickt es als Header `X-XSRF-TOKEN` bei verändernden Anfragen an relative URLs. Einstellbar mit `withXsrfConfiguration(...)`.
- **CORS** *(Kors)*: Regelt, **welche Webseiten Antworten lesen dürfen**. Es schützt nicht den Server und ist keine Absicherung.
- **HTTPS:** Verschlüsselte Verbindung. Immer verwenden. Keine sensiblen Daten in URLs.
- **Offene Weiterleitung (Open Redirect):** Die App leitet ungeprüft zu einer Adresse aus der URL weiter (z. B. `?returnUrl=`). Ein Angreifer kann so auf eine fremde Seite lenken. Gegenmittel: Ziel gegen eine **Allowlist** prüfen (*Alau-List*, Liste erlaubter Ziele), nur interne Pfade.
- **Unsichere Abhängigkeiten:** Alte Pakete mit Lücken. Hilfen: `npm audit` *(En-Pi-Em Ah-dit)*, Updates, Lockfile (festgehaltene Versionen), Dependabot/Renovate (automatische Update-Vorschläge).
- **Geheimnisse im Bundle:** Alles in `environment.ts` landet im Browser. Keine geheimen Schlüssel dort.
- **Datensparsamkeit:** Nur anzeigen und speichern, was gebraucht wird (Bezug zur DSGVO).
- **Zu viele Daten im Client:** Das Backend liefert nur erlaubte Felder. Was nur „versteckt" wird, ist trotzdem sichtbar.
- **Sensible Daten in Logs:** Keine Tokens oder Personendaten loggen (auch nicht im Interceptor).
- **Clickjacking** *(Klick-dschäking)*: Die Seite steckt unsichtbar in einem fremden Rahmen (`iframe`). Schutz am Server (`frame-ancestors`, `X-Frame-Options`).
- **CSP** *(Si-Es-Pi)* – **Content Security Policy**: Ein Server-Header, der festlegt, woher Skripte und Styles kommen dürfen. Begrenzt den Schaden bei XSS.
- **Trusted Types** *(Trass-ted Taips)*: Eine Browser-Funktion, die gefährliche DOM-Zuweisungen nur mit geprüften Werten erlaubt. Angular unterstützt sie.
- **Defense in Depth** *(Di-fens in Depth)*: „Mehrere Schutzschichten." CSP und Trusted Types ergänzen sauberes Escaping, ersetzen es aber nicht.
- **Subresource Integrity:** Prüfsumme für nachgeladene Fremd-Skripte, damit sie nicht unbemerkt verändert werden.
- **Fehlermeldungen:** Nicht zu detailreich (keine Stacktraces für Endnutzer).
- **Review-Checkliste Sicherheit:** `bypassSecurityTrust*`? `innerHTML` / `eval`? Ungeprüfte URL-Parameter? Tokens im Log oder `localStorage`? Geheimnisse im Bundle? Nur Guard statt Backend-Prüfung? Neue Abhängigkeiten mit `npm audit` geprüft?
- **Team-Aufgabe:** Sicherheit ist Sache des ganzen Teams: Review, Lint-Regeln, `npm audit` in der Pipeline, zeitnahe Updates mit `ng update`, dokumentierte Ausnahmen.

5.4 KI-gestützte Entwicklung (Kapitel 7, Übung 37)

- **KI-Assistent:** Ein **schneller Assistent ohne Verantwortung**. Die Verantwortung für den Code bleibt beim Menschen.
- **Gut geeignet:** Gerüste, Umbenennungen, Code erklären, Testideen, kleine Umbauten (mit Tests). **Nur als Gesprächspartner:** Architektur, Fachlogik, Sicherheit. **Riskant:** große Änderungen blind übernehmen.
- **Formen der Unterstützung: Chat** (Fragen stellen), **Inline-Vorschläge** in der Entwicklungsumgebung, **Agent**.
- **Agent** *(Ei-dschent)*: Ein KI-Werkzeug, das **selbstständig handelt**: Es liest das Projekt, ändert mehrere Dateien, führt Befehle und Tests aus. Mehr Freiheit heißt mehr Prüfaufwand und mehr möglicher Schaden.
- **Kontext:** Alles, was die KI gerade „weiß" (Aufgabe, Dateien, Regeln). Die KI kennt **nur**, was im Kontext steht. Gute Ergebnisse brauchen guten Kontext.
- **Projektregeln-Datei:** Eine Datei im Projekt mit Stil und Verboten (je nach Werkzeug z. B. `CLAUDE.md` oder `AGENTS.md`). Beispiel: „Angular 21, strict, zoneless, OnPush, kein `any`."
- **Offizielle Angular-Hinweise für KI:** Angular bietet Hilfen wie `llms.txt` und einen MCP-Server der Angular CLI an. Stand auf angular.dev prüfen. **MCP** *(Em-Si-Pi)* = Model Context Protocol, ein Weg, KI-Werkzeugen Zugriff auf Hilfsdienste zu geben.
- **Prompt** *(Prompt)*: Der Auftrag an die KI. Gutes Muster: **Kontext – Aufgabe – Vorgaben – Abnahmekriterien – Format.** Aufgabe klein und klar halten.
- **Halluzination** *(Ha-lu-zi-na-zi-on)*: Die KI erfindet etwas (Methoden, Optionen, Paketnamen), klingt aber überzeugend. Bild: *Ein Mitarbeiter, der lieber etwas erfindet, als „weiß ich nicht" zu sagen.*
- **Typische KI-Fehler bei Angular:** Veraltete Muster (`*ngFor`, `@Input()`, NgModule). Erfundene APIs. `effect` für alles. `any` und Typ-Casts. Unnötige Komplexität. Sicherheitsblindheit. Tests, die nichts prüfen. Stilbruch.
- **Review-Checkliste für KI-Vorschläge:** 1. Läuft es (Build, Lint, Tests)? 2. Passt es zur Architektur? 3. Ist es modern (keine veralteten APIs)? 4. Ist es sicher? 5. Ist es reaktiv korrekt? 6. Ist es getestet? 7. Verstehe ich es? **Was ich nicht erklären kann, committe ich nicht.**
- **Mehrrunden-Zusammenarbeit:** Mit der KI in mehreren Runden arbeiten: Ziel und Kontext, **erst Plan prüfen**, kleiner Schritt umsetzen, Review und Tests, Korrektur, nächste Runde. Ergebnisse (Fehlermeldungen, Testausgabe) an die KI zurückspielen.
- **Kontext pflegen:** In langen Sitzungen verblassen frühere Vorgaben. Regeln wiederholen oder neue Sitzung mit Zusammenfassung starten.
- **Erst Plan, dann Code:** Plan als Liste von Schritten besprechen, bevor Dateien geändert werden. Ein Commit pro Schritt.
- **Grenzen:** „Es kompiliert" heißt nicht „es ist richtig". „Klingt sicher" heißt nicht „stimmt". Der Wissensstand der KI kann veraltet sein. Wer nichts versteht, kann Ergebnisse nicht beurteilen.
- **Datenschutz** *(Da-ten-schutz)*: Keine Geheimnisse, Zugangsdaten oder Kundendaten in Prompts oder Kontextdateien. Firmenrichtlinien beachten (erlaubte Werkzeuge).
- **Lizenzen:** Die Herkunft von KI-Code kann unklar sein. Bei längeren Übernahmen Richtlinien beachten.
- **Vorgeschlagene Pakete prüfen:** Gibt es das Paket wirklich? Ist es gepflegt und sicher? Manchmal werden Paketnamen erfunden.
- **Bleibt Entwicklerkompetenz:** Anforderungen verstehen, Architektur abwägen, Code lesen und beurteilen, testen, messen, absichern, Verantwortung tragen.

5.5 Abschlussprojekt (Übung 38)

- **Abschlussprojekt „Kunden-Dashboard":** Neues Feature in Kleingruppen: Seite mit Kennzahlen (Anzahl Kunden, aktive Kunden der letzten 7 Tage, Kunden pro Stadt) unter der Route `/dashboard`. Wenn gewünscht, mit KI-Unterstützung.
- **Kennzahlen mit `computed`:** Alle Zahlen werden aus den Kundendaten **abgeleitet**, nicht extra gespeichert.
- **Daten über den `CustomerStore`**, die Seite als **Lazy Route** laden, keine unsicheren Muster, mindestens 3 sinnvolle Tests.
- **Vorgehen:** Plan (5 Schritte) im Team abstimmen, kleine Schritte, ein Commit pro Schritt.

- **Selbst-Review:** Mit den Checklisten (Aufteilen, Tests, Sicherheit, KI) durchgehen. Kurzvorstellung: Was hat die KI gut gemacht, was musste korrigiert werden?
- **Bewertet werden:** Architektur, Reaktivität, Tests, Sicherheit, Nachvollziehbarkeit des Vorgehens.

🔤 Alphabetisches Schnell-Nachschlagewerk (Quick Lookup)

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
Abgeleiteter / lokaler Zustand	*Ab-ge-lei-te-ter Zustand*	Lokal: der Nutzer ändert ihn direkt. Abgeleitet: wird per `computed` berechnet.
Abhängigkeitsrichtung	*Abhängigkeits-Richtung*	Abhängigkeiten zeigen nur nach unten (Komponente → Service → API), nie zurück.
Action (NgRx)	*Äk-schen*	Beschreibt die Absicht einer Zustandsänderung, z. B. „Kunde hinzufügen“.
Agent (KI)	*Ei-dschent*	KI-Werkzeug, das selbstständig Dateien ändert und Befehle ausführt.
Angular DevTools	*Ang-gju-lar Dev-Tuhls*	Browser-Erweiterung mit Komponentenbaum und Profiler.
AppComponent	*Äpp-Komponent*	Die Hauptkomponente (Wurzel) der gesamten Angular-Anwendung.
Arrange – Act – Assert	*Ä-reindsch – Äkt – Ä-sört*	Aufbau eines Tests: vorbereiten, ausführen, Ergebnis prüfen.
asReadonly	`asReadonly()`	Macht ein Signal von außen nur lesbar.
authGuard	`authGuard`	Beispiel-Guard der Übung: schickt nicht eingeloggte Nutzer zur Login-Seite.
Banana in a Box	`[()]`	Schreibweise für zweiseitige Datenbindung bei `[(ngModel)]`.
Bibliothek (Library)	*Lai-bre-ri*	Werkzeugkasten für eine Aufgabe, z. B. nur Rendering (React).
Binding	*Bain-ding*	Die Verknüpfung zwischen TypeScript-Variable und HTML-Anzeige.
bootstrapApplication	`bootstrapApplication(...)`	Startfunktion von Angular beim Laden der Webseite.
Budget	*Badd-schet*	Obergrenze für die Bundle-Größe. Wird sie überschritten, schlägt der Build fehl.
Bundle	*Bandl*	Gebautes Paket mit dem App-Code, das der Browser lädt.
bypassSecurityTrust	*Bai-päss Si-kjuh-ri-ti Trast*	Schaltet Angulars Schutz ab. Review-Alarm, nie für Nutzereingaben.
canActivate	`canActivate: [authGuard]`	Hängt einen Guard an eine Route.
Change Detection	*Tscheindsch Di-tek-schen*	Angular prüft, was sich geändert hat, und zeichnet es neu.
CLS	*Si-El-Es*	Cumulative Layout Shift: Springt das Layout?

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
computed	`computed(() => ...)`	Berechnetes Signal, das automatisch aktualisiert wird.
Container-Komponente	*Kon-tej-ner*	Kennt Services, lädt Daten und gibt sie an Darstellungskomponenten weiter.
Content Projection	*Kon-tent Pro-dschek-schen*	Eine Komponente zeigt Inhalt an, den sie von außen bekommt (`<ng-content>`).
contentChild	`contentChild()`	Signal auf ein von außen eingefügtes Element.
Core Web Vitals	*Kor Web Wai-tels*	Messwerte für das Nutzergefühl: LCP, INP, CLS.
CORS	*Kors*	Regelt, welche Webseiten Antworten lesen dürfen. Keine Absicherung des Servers.
Coverage	*Kaw-ridsch*	Testabdeckung. Hinweis, kein Ziel.
CSP	*Si-Es-Pi*	Content Security Policy: Server-Regel, woher Skripte geladen werden dürfen.
CSRF / XSRF	*Si-Es-Ar-Ef*	Fremde Seite löst mit dem Browser des Nutzers Aktionen aus. Schutz: SameSite und Token.
Customer	*Kastemer*	Kunde / Datenmodell für Kunden in den Schulungsübungen.
Darstellungskomponente	*Dar-stel-lungs-Komponente*	Kennt nur `input()` und `output()`. Zeigt an und meldet Ereignisse.
Datenschutz (KI)	*Da-ten-schutz*	Keine Geheimnisse oder Kundendaten in Prompts oder Kontextdateien.
DatePipe	*Deit-Paip*	Formatiert Datumsangaben im HTML lesbar.
Debounce	*Di-baunß*	Kurz warten nach einer Eingabe, bevor gerechnet wird.
Defense in Depth	*Di-fens in Depth*	Mehrere Schutzschichten statt einer.
@defer	*Di-för*	Lädt schwere Teile der Seite erst bei Bedarf.
Dependency Injection	*DI*	Automatisches Bereitstellen von Diensten und Klassen.
dirty / touched / invalid	`dirty` `touched` `invalid`	Feldzustände: geändert / besucht / Prüfregel verletzt.
E2E-Test	*I-tu-I-Test*	Ein Roboter bedient die ganze App im Browser.
effect	*I-fekt*	Führt eine Nebenwirkung nach außen aus, wenn sich Signals ändern.
esbuild / Vite	*I-Es-Bild / Wiet*	Schnelle Bau-Werkzeuge unter der Haube von Angular.
Event	*I-went*	Ereignis im Browser, z. B. Mausklick oder Tastendruck.
FormGroup	`FormGroup`	Gruppe von Formularfeldern in reaktiven Formularen (Wahlthema).

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
Framework	*Frejm-wörk*	Fertiger Baukasten mit Regeln. Angular bringt vieles gleich mit.
God Component	*Gott Komponent*	Riesen-Komponente, die alles macht. Schwer testbar.
Guard	*Gard*	Türsteher: entscheidet, ob eine Route geöffnet werden darf. Kein echter Schutz.
Halluzination	*Ha-lu-zi-na-zi-on*	Die KI erfindet etwas und klingt dabei überzeugend.
HttpClient	`HttpClient`	Angular-Dienst für Kommunikation mit Web-Servern.
HttpOnly-Cookie	*Ejtsch-ti-ti-pi-Ohn-li*	Cookie, den Script nicht auslesen kann.
httpResource	*Ejtsch-ti-ti-pi Ri-sors*	Lädt Daten per HTTP GET und liefert Signals (`value`, `isLoading`, `error`).
HttpTestingController	*Ejtsch-ti-ti-pi Testing Kontroller*	Fängt HTTP-Aufrufe im Test ab und liefert selbst gewählte Antworten.
Hydration	*Hai-drei-schen*	Der Browser belebt das vom Server gelieferte HTML wieder mit Angular.
inject	`inject(...)`	Funktion zum Anfordern eines Dienstes per Dependency Injection.
INP	*Ai-En-Pi*	Interaction to Next Paint: Wie schnell reagiert die App?
input	`input()`	Empfängt Daten von der übergeordneten Elternkomponente.
Interceptor	*Inter-sep-ter*	Fängt jede HTTP-Anfrage ab, z. B. zum Protokollieren.
Interpolation	`{{ ... }}`	Ausgabe von Variablen direkt im HTML-Text.
Komponententest	*Kom-po-nen-ten-Test*	Prüft Template und Klasse einer Komponente zusammen (`TestBed`).
Komposition	*Kom-po-zi-zi-on*	Kleine Bausteine zusammenstecken statt vererben.
Kontext (KI)	*Kon-tekst*	Alles, was die KI gerade weiß: Aufgabe, Dateien, Regeln.
Lazy Loading	*Lei-si Loh-ding*	Code oder Bilder erst laden, wenn man sie braucht.
LCP	*El-Si-Pi*	Largest Contentful Paint: Wann ist der Hauptinhalt sichtbar?
Lighthouse	*Lait-haus*	Messwerkzeug für Ladeverhalten und Core Web Vitals.
linkedSignal	*Linkt-Signal*	Abgeleiteter Wert, der trotzdem überschrieben werden kann.
loggingInterceptor	`loggingInterceptor`	Beispiel-Interceptor der Übung: protokolliert alle Anfragen.
LTS	*El-Ti-Es*	Long Term Support: Langzeit-Unterstützung einer Version.

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
Mehrrunden-Zusammenarbeit	*Mehr-run-den-Zu-sam-men-ar-beit*	Mit der KI in kleinen Runden arbeiten: Plan, Schritt, Review, Korrektur.
Meta-Framework	*Meta-Frejm-wörk*	Baut auf einem Framework auf und gibt noch mehr vor (Next.js, Nuxt).
Mock / Stub / Spy / Fake	*Mock / Stabb / Spai / Feik*	Ersatz-Objekte im Test, um Abhängigkeiten zu isolieren.
Mutation vs. neue Referenz	*Mjuh-tei-schen*	Ein Array direkt ändern bemerkt das Signal nicht. Ein neues Array erzeugen.
Nebenwirkung (Side Effect)	*Said I-fekt*	Etwas ändert sich außerhalb des Signal-Graphen, z. B. Speicher oder Logging.
ng serve	*en-dschi sörw*	Terminal-Befehl zum Starten des lokalen Entwicklungsservers.
ng update	*en-dschi ap-deit*	Befehl, der das Projekt auf die nächste Angular-Version hebt.
ng-content	`<ng-content>`	Platzhalter (Slot) für Inhalt, der von außen kommt.
ngOnInit	`ngOnInit()`	Lebenszyklus-Methode beim Initialisieren einer Komponente.
NgOptimizedImage	*En-Dschi Opti-maisd Imidsch*	Direktive für schnellere Bilder (`ngSrc`, `width`, `height`, `priority`).
NgRx	*En-Dschi-Ar-Ex*	Zustands-Bibliothek nach dem Redux-Muster.
Normalisierung	*Nor-ma-li-zie-rung*	Objekte einmal nach `id` ablegen, Listen enthalten nur IDs.
npm audit	*En-Pi-Em Ah-dit*	Prüft installierte Pakete auf bekannte Sicherheitslücken.
Observable	*Ob-sör-wa-bl*	Datenstrom, der über die Zeit Werte liefert (RxJS).
Offene Weiterleitung	*Open Redirect*	App leitet ungeprüft auf eine Adresse aus der URL weiter. Gegenmittel: Allowlist.
OnPush	*On-Pusch*	Komponente wird nur bei geänderten Inputs, Ereignissen oder Signals geprüft.
output	`output()`	Sendet Ereignisse von der Kind- an die Elternkomponente.
Pipe	`|`	Formatiert Daten direkt im HTML-Template.
Prerendering	*Pri-Ren-de-ring*	HTML wird schon beim Bauen erzeugt (SSG).
Prompt	*Prompt*	Der Auftrag an die KI. Muster: Kontext, Aufgabe, Vorgaben, Abnahme, Format.
Prop Drilling	*Prop Drilling*	Daten werden durch viele Ebenen durchgereicht, obwohl nur die letzte sie braucht.
Reactive Forms	*Ri-äktiv Forms*	Typsichere, im TypeScript-Code definierte Formulare (Wahlthema Z1).
Reducer (NgRx)	*Ri-djuh-ser*	Berechnet aus altem Zustand und Action den neuen Zustand.

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
Resource API	*Ri-sors*	Signal-basiertes asynchrones Laden (`resource`, `httpResource`, `rxResource`).
Review-Checkliste	*Ri-wju-Check-liste*	Liste zum Prüfen von Code, besonders bei KI-Vorschlägen und Sicherheit.
RouterOutlet	`<router-outlet>`	Platzhalter im HTML, wo die geroutete Seite erscheint.
Routes	*Rauts*	Liste der URL-Pfad-Zuordnungen der Anwendung.
rxResource	*Ar-Ex Ri-sors*	Resource mit RxJS-Observable, lädt bei geändertem `params` neu.
Sanitizing	*Sä-ni-tai-sing*	Gefährliche Teile aus HTML entfernen. Angular macht das bei `[innerHTML]`.
Selector (NgRx)	*Si-lek-tor*	Ableitung aus dem Zustand (im Signal Store: `withComputed`).
Service	*Sör-wis*	Klasse für Geschäftslogik und Datenabruf.
Signal	`signal()`	Reaktiver Datenspeicher mit automatischer Benutzeroberflächen-Aktualisierung.
Signal Query	*Signal Kwiri*	Sammelname für `viewChild`, `viewChildren`, `contentChild`, `contentChildren`.
Signal Store	*Signal Stohr*	`signalStore` aus `@ngrx/signals`: standardisierter Store mit Signals.
Single Page App	*SPA*	Webseite, die ohne Neuladen blitzschnell Seiten austauscht.
Single Responsibility	*Single Ri-spon-si-bi-li-ti*	Eine Aufgabe pro Baustein, nur ein Grund zur Änderung.
Single Source of Truth	*Single Sors of Truth*	Jeder Fakt wird nur an einer Stelle gespeichert.
SSR	*Es-Es-Ar*	Server-Side Rendering: Der Server liefert fertiges HTML.
Standalone	*Ständ-ä-lon*	Selbstständige Komponenten ohne `@NgModule`.
Store	*Stohr*	Zentrale Lagerhalle für Zustand, Ableitungen und Änderungs-Methoden.
Template	*Täm-pleit*	Die HTML-Vorlage einer Komponente.
TestBed	*Test-Bed*	Prüfstand von Angular für Tests.
Testpyramide	*Test-Pü-ra-mi-de*	Viele schnelle Unit-Tests, wenige langsame E2E-Tests.
toSignal	`toSignal(...)`	Wandelt ein RxJS-Observable in ein Angular-Signal um.
track	`track item.id`	Pflicht-Identifikator in `@for`-Schleifen für Performance.
Tree-Shaking	*Trie-Schei-king*	Der Build wirft ungenutzten Code weg.

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
Trusted Types	*Trass-ted Taips*	Browser-Funktion, die gefährliche DOM-Zuweisungen einschränkt.
TypeScript	*Teip-Skript*	Programmiersprache von Angular mit strenger Typisierung.
Unit-Test	*Juh-nit-Test*	Prüft eine kleine Einheit, z. B. Service, Store oder pure Funktion.
untracked	*An-trakt*	Liest ein Signal, ohne davon abhängig zu werden.
Vertrauensgrenze	*Fer-trau-ens-gren-ze*	Grenze zwischen eigenem Code und Eingaben von außen. Alles von außen wird geprüft.
viewChild	*Wju-Tschaild*	Signal auf ein Element im eigenen Template (`#name`).
Virtual Scrolling	*Wör-tschu-el Skrol-ling*	Bei langen Listen sind nur die sichtbaren Zeilen im Dokument.
Vitest	*Wih-test*	Test-Werkzeug, in Angular 21 Standard.
XSS	*Iks-Es-Es*	Cross-Site Scripting: Fremder Script-Code läuft in der Seite.
Zoneless	*Sohn-les*	Moderner Signal-Modus ohne veraltete `zone.js`-Überwachung.
Zustandsverwaltung	*Zu-stands-ver-wal-tung*	Wie und wo eine App ihre Daten hält und ändert.
Zyklus (Signals)	*Zü-klus*	Effects schreiben einander gegenseitig: Endlosschleife. Besser eine Quelle plus `computed`.